

SECURITY REVIEW

LI.FI



GETRECON.XYZ
@GALLODASBALLO
@OXKNOT

LI.FI Security Review

Introduction

Alex the Entrepreneur and Knot performed 2 weeks of Invariant Testing on the LI.FI VirtualMachine.

Repo: <https://github.com/lifinance/Yggdrasil-VirtualMachine/>

Commit: b0458e9dac9fc99ca74f3fabadf406fe5bc0cced

The Review is done as a best effort service, while a lot of time and attention was dedicated to the security review, it cannot guarantee that no bug is left

As a general rule we always recommend doing one additional security review until no bugs are found, this in conjunction with a Guarded Launch and a Bug Bounty can help further reduce the likelihood that any specific bug was missed

The codebase maturity was high, given the high specificity of the possible issues we highly recommend performing a Security Contest or a Bug Bounty, with an explicit list of integrations that will be used as to highlight any possible integration risk.

About Recon

Recon offers boutique security reviews, invariant testing development and is pioneering Cloud Fuzzing as a best practice by offering Recon Pro, the most complete tool to run tools such as Echidna, Medusa, Foundry, Kontrol and Halmos in the cloud with just a few clicks

About Alex

Alex is the cofounder of Recon and a well known Lead Security Researcher with \$500,000 in bounties and contest winnings.

- Code4rena - One of the most prolific and respected judges, won the Tapioca contest, at the time the 3rd highest contest pot ever
- Spearbit - Have done reviews for Tapioca, Threshold USD, Velodrome and more
- Recon - Centrifuge Invariant Testing Suite, Corn and Badger invariants as well as live monitoring

About Knot

Knot is the creator of the Recon Extension.

Senior Full-Stack Developer, Lead Invariants Engineer, driving R&D and future tools at Recon.

Additional Services by Recon

Recon offers:

- Audits powered by Invariant Testing - We'll write your invariant tests then perform an audit on your code.
- Cloud Fuzzing as a Service - The easiest way to run invariant tests in the cloud - Ask about Recon Pro.
- Invariant Tests writing - An engineer will write Chimera based Invariant Tests on your codebase.

Table of Contents

- **Invariants**

- I-01 Properties Tested

- **Med**

- M-01 Dynamic tuple element pointer base off by 0x20 (array elements)
- M-02 Dynamic container metadata ordering causes sibling mispairing
- M-03 Dynamic array element pointer base off by 0x20
- M-04 Discrepancy of abi.encode when using just an empty array
- M-05 `depositApproved` cannot work with USDT and ERC20 that don't return a `bool` on `transfer`

- **QA**

- Q-01 Commands default to OP call
- Q-02 Incomplete list of Memory Unsafe Operations
- Q-03 Gotcha: Transferring tokens will always require checking the return value
- Q-04 `depositApproved` can return the incorrect amount for FoT and stETH like tokens
- Q-05 `depositApproved` can return 0 on non existent tokens

- **Analysis**

- A-01 Suggested Next Steps
- A-02 Integration risks inspite of presence of slippage check when an attacker can regain access
- A-03 Incomplete list of Reentrancy Techniques
- A-04 RPN Math Gotchas
- A-05 Reflections on using the VM in production
- A-06 Handpicked Chain Specific Integration Risks

I-01 Properties Tested

Halmos

The halmos tests provide a baseline for most of the Libraries.

The only limitation is that some specific tests take a very long time to run when `--loop` goes past 3.

Most of the FV tests ensure that the libraries are well behaved.

Leaving mostly combinational risks (Free Memory Pointer related issues), and Integrational Risks (reentrancy, incorrect integration)

The `DepositApprovedHalmosTest` shows a fairly concise yet extensive way to test for ERC20 behavior.

ArithmeticProcessorHalmosTest

- `check_evaluateRPN` : Evaluates the RPN stream against an unoptimized reference
- `check_fuzzRPN_doesn_revert` : TODO: Failing
- `check_oneOpFuzzRPN` : Performs one operation in a templated symbolic way and compares against doing it locally (e.g: `a + b`)
- `check_rpnLenZeroAlwaysReverts` : Shows that `rpnLen` set to 0 always reverts
- `check_maxItemsInStack` : debug test that shows the max items in the stack (16)
- `test_simple2` repro for a edge case

BlueprintEncoderHalmosTest

- `check_symbolic_flat_matches_abi()` - Simple symbolic flat blueprint (`encodeData`)
- `check_encode_from_blueprint_flat_matches_abi()` - Simple symbolic flat blueprint (`encodeFromBlueprint`)
- `check_static_array2_matches_abi()` - Simple static 2 element array, static elements
- `check_static_array2_with_dynamic_elements_matches_abi()` - Simple static 2 element array, dynamic elements
- `check_dynamic_array_matches_abi()` - Simple dynamic 2 element array, static elements
- `check_dynamic_array_with_dynamic_elements_matches_abi()` - Simple dynamic 2 element array, dynamic elements
- `check_tuple_with_static_elements_matches_abi()` - Simple 2 elements tuple encoding, static elements
- `check_tuple_with_dynamic_elements_matches_abi()` - Simple 2 elements tuple encoding, dynamic elements
- `check_bad_static_format_reverts()` - Invalid static length reverts
- `check_bad_dynamic_format_reverts()` - Items with <32 length reverts for Dynamic elements
- `check_payload_length_invariants()` - `EncodeData` and `encodeFromBlueprint` have the correct lengths (X, X+4)
// TODO
- `check_pointer_bounds_flat()` - All head pointers are 32-byte aligned and point inside the buffer.
- `check_pointer_alignment_dynamic_array()` - All head pointers are 32-byte aligned and point inside the buffer.
- `check_pointer_alignment_nested_tuple()` - All head pointers are 32-byte aligned and point inside the buffer.
- `check_generative_encoder_invariants()` - Differential FV of `encodeData` and `encodeFromBlueprint` (Limited)
- `check_unclosed_container_reverts()` - An unmatched closed container always reverts
- `check_stack_overflow_reverts()` - Stack overflow always reverts
- `check_empty_blueprint_returns_empty()` - Empty blueprint returns empty bytes
- `check_missing_register_static_reverts()` - OOB access reverts
- `check_missing_register_dynamic_reverts()` - OOB Dynamic access reverts
- `check_vm_abi_encode_equals_encoder()` - VM ABI_ENCODE(bp) output equals `BlueprintEncoder.encodeData(bp, regs)` (Limited)
- `check_vm_calldata_build_equals_concat()` - VM CALLDATA_BUILD(sel, bp) output equals `bytes.concat(sel, ABI_ENCODE(bp))` (Limited)

BlueprintEncoderHalmosTest Source Altering Tests

We cannot merge these as they alter the library <https://github.com/Recon-Fuzz/lifi-vm-invariants/pull/2>

These tests show that values that are too high can overflow and significantly alter the behaviour of the Blueprint encoder.

I recommend reviewing these and showing them to auditors.

CommandPackingHalmosTest

Encodes a fully symbolic `Operation` and verifies that `unpacking` returns the original values

- `check_call_roundtrip`
- `check_calldata_build_roundtrip`
- `check_explode_roundtrip`
- `check_deposit_approved_roundtrip`
- `check_surgery_roundtrip`
- `check_return_roundtrip`
- `check_abi_encode_roundtrip`
- `check_remaining_gas_roundtrip`
- `check_native_balance_roundtrip`
- `check_log_roundtrip`

EDGE CASES:

- `check_calldata_build_blueprint_too_large_reverts`
- `check_abi_encode_blueprint_too_large_reverts`
- `check_unpackCallDataBuild_overflow`
- `check_call_packed_roundtrip` | NOTE: Reverse roundtrips should cause issues due to dirty lower bits

DepositApprovedHalmosTest

- `check_depositApprovedWithCustomBehavior` Fully symbolic test with ERC20 with custom behaviour. Also assert that a non existing token should cause a revert.

Clamped handlers

- `check_deposit_approved_command` - Simple deposit flow with symbolic approve and mint amounts
- `check_deposit_approved_zero_approval` - 0 Approval should result in zero transfers
- `check_deposit_approved_zero_balance` - 0 balances should result in zero transfers
- `check_deposit_approved_invalid_token` - Non existing token should not be transferred and should cause a revert

InvariantCheckerHalmosTest

- `check_single_op`
- `check_batch_vs_packed`
- `check_public_asserts_match_internal`
- `check_packOps_boundaries_and_manual`
- `check_batchPacked_too_many`
- `check_public_asserts_match_internal`
- `check_inRange_inclusive_bounds`
- `check_inRange_min_gt_max_reverts_ge_min`
- `check_inRange_min_gt_max_reverts_lt_min`
- `check_nop_no_advance_in_batch`
- `check_batchPacked_boundaries`
- `check_packOps_boundaries_and_manual`

RegisterFileHalmosTest:

- `check_get_static` - Simple getter with symbolic values
- `check_getStatic` - Simple getter with symbolic value for `getStatic`
- `check_set` - Verifies `set` works symbolically
- `check_setDynamic` - Verifies `setDynamic` works symbolically
- `check_set_then_get` - Chains a set and then a get symbolically
- `check_get_oob_reverts` - Check revert for OOB on get
- `check_set_oob_reverts` - Check revert for OOB on set
- `check_get_void_reg_always_zero` - Ensures the void registry always returns 0
- `check_set_void_reg_noop` - Ensures that setting a value to the void registry is a no-op
- `check_setDynamic_short_payload_reverts` - Ensures symbolically that bytes that are too short cannot be `setDynamic`

SurgeryOPSHalmosTest

`check_perform_surgery` Fully symbolic test that verifies:

- The length of the original bytes remains consistent with the length of the bytes returned
- Bytes outside of the Surgery area should remain unchanged
- The Result matches a local implementation

NOTE: This test can take a very long time It should terminate quickly with `--loop 3` However for `--loop 6` and higher this seems to take more than an hour

Medusa

Latest Run: <https://getrecon.xyz/shares/7f483082-ffeb-4589-bf79-8bc1e0531fcf>

The medusa tester proved to be very limited in what it was able to test without requiring extensive clamping.

Due to this, the properties it is able to verify are very broad and in many cases dependent on the input.

This stems from the limitations that Solidity impose (as we can't enumerate a contracts selectors programmatically), as well as how Medusa handles it's dictionary (making it hard to link the bytes32 of a blueprint to an `abi.encode` process)

Due to this, we recommend the Medusa suite is used mostly to asses high level integrations, and suggest instead using the Custom Tester to asses specific libraries.

High Level Properties

- `invariant_registries` - Taints registries past index 128 and verifies that these were never altered
- `property_untouched_registries` - After a clamped call (which uses state in storage) it inspects the commands used and ensures that only the registries marked as destination have been altered

Tightly Coupled Properties

`doomsday_depositApprovedExactAmountTest` - Checks that `DEPOSIT_APPROVED` will behave as intended then returns the value for inspection. This currently fails per the various edge cases we highlighted.

`doomsday_callReturnsTheCorrectValue` - Performs a call to a clamped target and verifies that the `CALL` behaves as intended. (Uses the `srcReg`, returns data to `callReturnReg` and the data is correctly `abi.encode`d). Has an add

`doomsday_gas_in_register` - Performs 2 `REMAINING_GAS` operations to assert that: the gas is monotonically decreasing, it will be returned in the correct register, it will actually alter the value, and that the call should never revert unless it was provided insufficient gas. It then performs one last call to

ensure that when `runWithState` is provided with an exact gas amount, the `REMAINING_GAS` operation will return a value that is strictly lower than that.

Custom Tester

Concrete testing for `BlueprintEncoder`, `RPN` and `SurgeryOps`.

The custom tester uses real data to generate test cases, then uses foundry to execute them, check the README for extensive documentation:

https://github.com/Recon-Fuzz/li-fi_bf

From empirical evidence this custom tool has resulted in the best results for the `BlueprintEncoder`, `RPN` and `SurgeryOps`.

We highly recommend running this after every major change.

M-01 Dynamic tuple element pointer base off by 0x20 (array elements)

Summary

When the element of a dynamic array is itself a dynamic tuple, the pointer emitted at container open was computed relative to the array head start, not accounting for the 32-byte array length word. This produced a systematic +0x20 error in element head pointers. While F-01 fixed the pointer base for dynamic registers inside arrays, the container-open path for dynamic tuple elements still used the wrong base.

Impact

- Incorrect calldata for dynamic arrays of dynamic tuples, e.g. orders-style payloads with bytes fields.
- Affected real-world shapes such as `getOrderFillableMakerAssetAmounts(...,bytes[])` and similar.

Affected Components

- `BlueprintEncoder._encodeCore` when opening a `DYNAMIC_TUPLE` child while the current frame is a `DYNAMIC_ARRAY` (pointer write site at container open).

Proof of Concept

- Simplified function: `f((address,uint256,bytes,bytes)[],bytes[]) (selector = bytes4(keccak256("f((address,uint256,bytes,bytes)[],bytes[])")) )`.
- Blueprint DSL (hex): `0x7c7e000182837b7b7c84857b`
 - `[ pushArray, pushTuple, s(0), s(1), d(2), d(3), endTuple, endArray, pushArray, d(4), d(5), endArray ]`
- Registers (`RegisterFile.initialize(6)`):
 - `regs[0] = abi.encode(address(0x1111...1111))`
 - `regs[1] = abi.encode(uint256(123))`
 - `regs.setDynamic(2, abi.encode(bytes("0xdead")))`
 - `regs.setDynamic(3, abi.encode(bytes("beef")))`
 - `regs.setDynamic(4, abi.encode(bytes(hex"5b59f0")))`
 - `regs.setDynamic(5, abi.encode(bytes(hex"01bef3b2")))`

```

struct OrderLite {
    address maker;
    uint256 amount;
    bytes data1;
    bytes data2;
}

function test_poC_ordersByteArray() public {
    // Registers layout (by index):
    // 0: order[0].maker (static)
    // 1: order[0].amount (static)
    // 2: order[0].data1 (dynamic)
    // 3: order[0].data2 (dynamic)
    // 4: sigs[0] (dynamic)
    // 5: sigs[1] (dynamic)
    bytes[] memory regs = RegisterFile.initialize(6);

    regs[0] = abi.encode(
        address(0x1111111111111111111111111111111111111111)
    );
    regs[1] = abi.encode(uint256(123));
    regs.setDynamic(2, abi.encode(bytes("0xdead")));
    regs.setDynamic(3, abi.encode(bytes("beef")));
    regs.setDynamic(4, abi.encode(bytes(hex"5b59f0")));
    regs.setDynamic(5, abi.encode(bytes(hex"01bef3b2")));

    // Blueprint:
    // arg0: orders: dynamic array of dynamic tuple (address,uint256,bytes,bytes)
    //   [ pushArray, pushTuple, s(0), s(1), d(2), d(3), endTuple, endArray ]
    // arg1: sigs: dynamic array of bytes
    //   [ pushArray, d(4), d(5), endArray ]
    bytes memory bp;
    // orders
    bp = bytes.concat(Bp.pushArray());
    bp = bytes.concat(bp, Bp.pushTuple());
    bp = bytes.concat(bp, Bp.s(0), Bp.s(1), Bp.d(2), Bp.d(3));
    bp = bytes.concat(bp, Bp.end()); // end tuple
    bp = bytes.concat(bp, Bp.end()); // end orders array
    // sigs
    bp = bytes.concat(bp, Bp.pushArray());
    bp = bytes.concat(bp, Bp.d(4), Bp.d(5));
    bp = bytes.concat(bp, Bp.end());

    // selector: f((address,uint256,bytes,bytes)[],bytes[])
    bytes4 selector = bytes4(
        keccak256("f((address,uint256,bytes,bytes)[],bytes[])")
    );

    // Encode via BlueprintEncoder
    bytes memory got = BlueprintEncoder.encodeFromBlueprint(
        selector,
        bp,
        regs
    );
    bytes memory gotStripped = TestUtils.stripLength(got);

    // Expected using abi.encodeWithSelector
    OrderLite[] memory orders = new OrderLite[](1);
    orders[0] = OrderLite({
        maker: address(0x1111111111111111111111111111111111111111),
        amount: 123,
        data1: bytes("0xdead"),
        data2: bytes("beef")
    });
    bytes[] memory sigs = new bytes[](2);
    sigs[0] = bytes(hex"5b59f0");
    sigs[1] = bytes(hex"01bef3b2");
    bytes memory exp = abi.encodeWithSelector(selector, orders, sigs);

    assertEq(gotStripped, exp, "PoC minimal: encoding mismatch");
}

```

Root Cause

The relative pointer base at the dynamic-container open site did not skip the array's 32-byte length word. This is analogous to F-01 but occurs in the code path that writes the pointer for a child dynamic container (tuple) inside a dynamic array, not in the dynamic-register path.

Remediation

At dynamic-container open, when the parent frame is a `DYNAMIC_ARRAY`, compute the relative pointer against `headStart + 32` (skip the array length) instead of `headStart`.

Patch ([Full fix can be found at (includes fix for #35 and #36)]
(<https://gist.github.com/0kn0t/2d633de493deead4e453a450ddd9ef1>))

Note: Diff below is illustrative/reference-only. The repository's current `src/BlueprintEncoder.sol` is the source of truth.

```
--- a/src/BlueprintEncoder.sol
+++ b/src/BlueprintEncoder.sol
@@ function _encodeCore(bytes calldata blueprint, bytes[] memory registers, uint256[] memory dynHead, bytes memory
out, uint256 startOffset, uint256 headBytes)
    if (ck == ContainerKind.DYNAMIC_TUPLE || ck == ContainerKind.DYNAMIC_ARRAY) {
        // This container is dynamic. Write a pointer to it in the parent's head.
        // The pointer's target is the current end of the parent's tail.
        uint256 absPtr = _fTailStart(frame) + _fTailOff(frame);
-       uint256 relPtr = absPtr - _fHeadStart(frame);
+       // If the parent is a dynamic array, element head pointers are relative
+       // to the array head AFTER the 32-byte length word.
+       uint256 base = _fHeadStart(frame);
+       if (_fKind(frame) == ContainerKind.DYNAMIC_ARRAY) {
+           unchecked { base += 32; }
+       }
+       uint256 relPtr = absPtr - base;
```

M-02 Dynamic container metadata ordering causes sibling mispairing

Summary

Dynamic container metadata (head sizes and, for arrays, element counts) was stored and consumed in mismatched orders. With sibling dynamic containers (e.g., `bytes[]` alongside `bool[]`), metadata could be associated with the wrong container, producing bad lengths/pointers.

Impact

- Incorrect calldata for functions with multiple sibling dynamic containers (e.g., `string, bytes[], bool[], address`).
- Symptoms observed as wrong array length and misaligned element pointers; breaks correctness and interoperability with ABI consumers.

Affected Components

- `BlueprintEncoder._measure` (metadata storage discipline)
- `BlueprintEncoder._encodeCore` (metadata consumption order)

Proof of Concept

- Function: `f1(string,bytes[],bool[],address)`
- Blueprint (bytes): `0x807c81827b7c037b04` (tokens: `d(0)`, `[pushArray d(1) d(2) end]`, `[pushArray s(3) end]`, `s(4)`).
- Registers: `string("hi")`, `bytes("A")`, `bytes("BC")`, `bool(false)`, `address(0x22..22)`
- Selector: `0x6390f115`
- Before fix: `bytes[]` head showed length `1` vs expected `2` and offset schedule deviations (sibling metadata mispairing).

Root Cause

Metadata was effectively LIFO-associated with dynamic containers; later, encoding consumed in the same reversed manner. With siblings, the pairing between containers and metadata drifted.

Remediation

- During measurement: when opening a dynamic container, reserve a metadata slot (START order). When closing it, write its metadata into that reserved slot.
- During encoding: consume metadata in START order (incrementing) to match traversal.

Patch ([Full fix can be found at (includes fix for #35 and #37)]
(<https://gist.github.com/0kn0t/2d633de493deead4e453a450ddd9ef1>))

Note: The following diffs are provided for review context and are illustrative/reference-only. They may be simplified and might not match the final fixed code verbatim.

Measurement pass (START-ordered metadata):

```
--- a/src/BlueprintEncoder.sol
+++ b/src/BlueprintEncoder.sol
@@ function _measure(bytes calldata blueprint, bytes[] memory registers, uint256[10] memory tmpDynHeads)
    uint256[MAX_STACK_DEPTH] memory stack;
    uint256[MAX_STACK_DEPTH] memory elemCount;
+   uint256[MAX_STACK_DEPTH] memory metaIndex; // START-ordered slots per open container
    uint256 sp; // stack pointer
+   uint256 nextMetaIdx; // next tmpDynHeads slot to assign on OPEN

@@
    if (open != ContainerKind.NONE) {
        if (sp + 1 >= MAX_STACK_DEPTH) revert StackOverflow();
        stack[++sp] = _mkSizing(0, 0, open);
        elemCount[sp] = 0;
+       if (open == ContainerKind.DYNAMIC_ARRAY || open == ContainerKind.DYNAMIC_TUPLE) {
+           if (nextMetaIdx == tmpDynHeads.length) revert DynHeadBufOverflow();
+           metaIndex[sp] = nextMetaIdx++;
+       }
    }

@@
    bool dynamic = (ck == ContainerKind.DYNAMIC_TUPLE || ck == ContainerKind.DYNAMIC_ARRAY);
    if (dynamic) {
        uint256 idx = metaIndex[sp + 1];
        if (ck == ContainerKind.DYNAMIC_ARRAY) {
            uint256 packed = childHeadLen | (childElemCount << SHIFT_CNT);
            tmpDynHeads[idx] = packed;
        } else {
            tmpDynHeads[idx] = childHeadLen;
        }
        stack[sp] = _szAddHead(stack[sp], 32);
        uint256 bytesChild = childHeadLen + childTailLen;
        if (ck == ContainerKind.DYNAMIC_ARRAY) {
            stack[sp] = _szAddTail(stack[sp], bytesChild + 32 /*length word*/ );
        } else {
            stack[sp] = _szAddTail(stack[sp], bytesChild);
        }
    } else {
        stack[sp] = _szAddHead(stack[sp], childHeadLen);
        stack[sp] = _szAddTail(stack[sp], childTailLen);
    }
}

@@
    headBytes = _szHead(stack[0]);
    totalBytes = headBytes + _szTail(stack[0]);
+   dynHeadCount = nextMetaIdx;
}
```

Encoding pass (START-ordered consumption):

```

--- a/src/BlueprintEncoder.sol
+++ b/src/BlueprintEncoder.sol
@@ function _encodeCore(bytes calldata blueprint, bytes[] memory registers, uint256[] memory dynHead, bytes memory
out, uint256 startOffset, uint256 headBytes)
    if (ck == ContainerKind.DYNAMIC_TUPLE || ck == ContainerKind.DYNAMIC_ARRAY) {
        // ... pointer write elided ...
        frame = _fBumpHead(frame, 32);

        // Retrieve pre-measured metadata in START order
        if (dynPop >= dynHead.length) revert DynHeadBufOverflow();
-       uint256 meta = dynHead[dynHead.length - 1 - dynPop++];
+       uint256 meta = dynHead[dynPop++];
        uint256 childHeadLen = meta & MASK_LEN;
        // ... encode child elided ...
    }

```

M-03 Dynamic array element pointer base off by 0x20

Summary

Element head pointers inside dynamic arrays of dynamic elements were computed relative to the start of the array head, not accounting for the 32-byte array length word, resulting in an extra 0x20 offset (e.g., 0x40 observed vs 0x20 expected for the first element).

Impact

- Incorrect calldata for dynamic arrays of dynamic elements: `string[]`, `bytes[]`, and arrays of dynamic tuples.
- Any consumer relying on the encoder would send malformed calldata, breaking contract interactions and tooling that parses ABI data.

Affected Components

- `BlueprintEncoder._encodeCore` when emitting pointers for dynamic registers while the active container is a `DYNAMIC_ARRAY`.

Proof of Concept

- Function under test: `f(string[]) (selector = bytes4(keccak256("f(string[])")))`
- Blueprint (DSL): `[ START_ARRAY_DYNAMIC, d(0), END ]` built as `abi.encodePacked(Bp.pushArray(), Bp.d(0), Bp.end())`
- Registers: `regs = RegisterFile.initialize(1); regs.setDynamic(0, abi.encode(string("x")));`
- Assertions (as in `test/BlueprintEncoderPoC.t.sol::test_poc_bug1_string_array_pointer_base()`):
 - `raw = BlueprintEncoder.encodeFromBlueprint(selector, bp, regs)`
 - `sigEnc = abi.encodeWithSignature("f(string[])", ["x"])`
 - Expect mismatch due to bytes memory leading length word: `keccak256(raw) != keccak256(sigEnc)`
 - After stripping the leading length word, expect equality: `striplength(raw) == sigEnc`

```
function test_poc_bug1_string_array_pointer_base() public pure {
    // registers: [0] dynamic string "x"
    bytes[] memory regs = RegisterFile.initialize(1);
    regs.setDynamic(0, abi.encode(string("x")));

    // blueprint: [ START_ARRAY_DYNAMIC, d(0), END ] => f(string[])
    bytes memory bp = abi.encodePacked(Bp.pushArray(), Bp.d(0), Bp.end());
    bytes4 selector = bytes4(keccak256("f(string[])"));

    bytes memory raw = BlueprintEncoder.encodeFromBlueprint(
        selector,
        bp,
        regs
    );

    string[] memory arr = new string[](1);
    arr[0] = "x";
    bytes memory sigEnc = abi.encodeWithSignature("f(string[])", arr);

    assertEq(
        TestUtils.striplength(raw),
        sigEnc,
        "PoC Bug1: stripped matches canonical signature encoding"
    );
}
```

Root Cause

Relative pointer base did not skip the array's 32-byte length word when writing element head pointers for dynamic arrays.

Remediation

When writing a dynamic register pointer inside a dynamic array frame, set the base as `headStart + 32` (skip length word) before computing the relative pointer.

Patch ([Full fix can be found at (includes fix for #36 and #37)] (<https://gist.github.com/0kn0t/2d633de493deeaad4e453a450ddd9ef1>))

Note: The following diffs are provided for review context and are illustrative/reference-only. They may be simplified and might not match the final fixed code verbatim.

```
--- a/src/BlueprintEncoder.sol
+++ b/src/BlueprintEncoder.sol
@@ function _encodeCore(bytes calldata blueprint, bytes[] memory registers, uint256[] memory dynHead, bytes memory
out, uint256 startOffset, uint256 headBytes)
    else if (tok.isDyn()) {
        uint8 idx = tok.idx();
        if (registers.get(idx).length < 32) revert BadDynamicFormat(idx);

        // Write pointer to head.
        uint256 absPtr = _fTailStart(frame) + _fTailOff(frame);
-        // The pointer value is relative to the start of the ABI data block.
-        uint256 relPtr = absPtr - _fHeadStart(frame);
+        // For dynamic arrays, element head pointers are relative to the start of the
+        // array head AFTER the 32-byte length word. For other containers, pointers are
+        // relative to the container's head start as usual.
+        uint256 base = _fHeadStart(frame);
+        if (_fKind(frame) == ContainerKind.DYNAMIC_ARRAY) {
+            unchecked { base += 32; } // skip array length word
+        }
+        uint256 relPtr = absPtr - base;
        _writeUint(out, _fHeadStart(frame) + _fHeadOff(frame), relPtr);

        // Copy data to tail.
        uint256 bytesW = _copyDyn(out, absPtr, registers.get(idx));
        // Advance head offset (for the pointer) and tail offset (for the data).
        fs[sp] = _fBumpHead(frame, 32);
        fs[sp] = _fBumpTail(fs[sp], bytesW);
    }
}
```

POC

Results

From Blueprint (trimmed)

[illegible]

From `abi.encode`

[illegible]

M-05 `depositApproved` cannot work with USDT and ERC20 that don't return a `bool` on `transfer`

Impact

`depositApproved` will perform the following:

<https://github.com/lifinance/VM-External-Audit-Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/DepositApproved.sol#L43-L51>

```
uint256 transferAmount = approvedAmount < userBalance ? approvedAmount : userBalance;
/// @audit MED - Doesn't work with USDT
bool transferred = token.transferFrom(msg.sender, address(this), transferAmount);
if (!transferred) {
    revert VmErrors.TokenTransferFailed();
}

return transferAmount;
}
```

`token.transferFrom` uses the standard ERC20 interface where `transferFrom` will return a `bool`

<https://github.com/lifinance/VM-External-Audit-Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/interfaces/IERC20.sol#L8-L9>

```
function transferFrom(address sender, address recipient, uint256 amount) external returns (bool);
```

Tokens such as `USDT` do not return a boolean, meaning the contract won't be able to interact with it

POC

```
// forge test --match-test test_doomsday_depositApprovedExactAmountTest_ -vvv

function test_doomsday_depositApprovedExactAmountTest_() public {

    switch_asset(0);

    doomsday_depositApprovedExactAmountTest(2026545506588193724530213580039430288999889098760471737131669757400780111838);
}
```

Mitigation

Use `safeTransferFrom` from Openzeppelin, Solmate or Solady.

Q-01 Commands default to OP call

Impact

Calling the VM with empty data causes it to revert at this line: <https://github.com/Recon-Fuzz/lifi-vm-invariants/blob/7f2e21107c1bd57af810cae031c3f6a3a19db90e/src/VirtualMachine.sol#L56-L62>

```
if (op == OP.CALL) {  
    // External contract invocation with configurable call semantics  
    Call memory c = CommandPacking.unpackCall(data);  
    bytes memory callData = vmState.registers.get(c.srcReg.idx());  
  
    // Check that calldata length is at least 36 bytes (32 for the length word + 4 bytes for the selector)  
    if (callData.length < 36) revert VmErrors.InvalidCallDataLength();  
}
```

This is because `op == 0` causes it to default to `OP.Call`

POC

Changing the `VMCommand` length to 2 causes the bug

[https://github.com/Recon-Fuzz/lifi-vm-](https://github.com/Recon-Fuzz/lifi-vm-invariants/blob/7f2e21107c1bd57af810cae031c3f6a3a19db90e/test/recon/targets/DoomsdayTargets.sol#L61-L99)

[invariants/blob/7f2e21107c1bd57af810cae031c3f6a3a19db90e/test/recon/targets/DoomsdayTargets.sol#L61-L99](https://github.com/Recon-Fuzz/lifi-vm-invariants/blob/7f2e21107c1bd57af810cae031c3f6a3a19db90e/test/recon/targets/DoomsdayTargets.sol#L61-L99)

```
``solidity
function doomsday_callReturnsTheCorrectValue(
    uint8 callReturnReg,
    uint8 srcReg,
    uint256 absoluteValue // NOTE: Can be extended to be an arbitrary value
) public stateless {

    require(keccak256(abi.encode(absolutValue)) != keccak256(DIRTY_REGISTRY_FLAG));

    VMState memory initialState;
    initialState.registers = new bytes[](256);
    for (uint256 i = 0; i < 256; i++) {
        initialState.registers[i] = DIRTY_REGISTRY_FLAG;
    }

    omniTarget.setValue(absolutValue);

    bytes memory initialCallValue = removeOffsetAssembly(abi.encode(abi.encodeCall(OmniTarget.returnValue, ()))));

    // Calldata
    initialState.registers[srcReg] = initialCallValue;

    VMCommand[] memory doomsdayCommands = new VMCommand[](2); /// @audit 2 lenght instead of 1
    doomsdayCommands[0] = VMCommand(OP.CALL, CommandPacking.packCall(address(omniTarget), uint8(CallType.CALL),
    callReturnReg, srcReg, 0));

    try virtualMachine.runWithState(doomsdayCommands, initialState) returns (VMState memory _state, bytes memory)
    {
        // Check that ssrcReg is unchanged
        // Check that dstReg has the value we expect
        t(keccak256(initialCallValue) == keccak256(_state.registers[srcReg]), 'Call src value is unchanged');
        try this.doAbiDecode(_state.registers[callReturnReg]) returns (uint256 decodedValue) {
            t(decodedValue == absoluteValue, 'Call return value is not correct');
        } catch {
            t(false, "doAbiDecode failed");
        }
    } catch {
        t(false, "Call should never revert with a known target");
    }
}
```

Mitigation

Consider shifting the enums by 1, or acknowledge this as a informational finding

Q-02 Incomplete list of Memory Unsafe Operations

VirtualMachine

[https://github.com/lifinance/VM-External-Audit-](https://github.com/lifinance/VM-External-Audit-Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/VirtualMachine.sol#L79-L84)

[Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/VirtualMachine.sol#L79-L84](https://github.com/lifinance/VM-External-Audit-Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/VirtualMachine.sol#L79-L84)

```
// Bubble up the exact revert data from the failed call
assembly { /// @audit this is not memory safe
    let size := returndatasize()
    returndatacopy(0x00, 0x00, size)
    revert(0x00, size)
}
```

The above snippet is considered memory unsafe because memory safety is an optimization step, since this will possibly allocate to the FMP as well as 0x60 (Zero Registry), this will result in undefined behavior if marked as safe.

Source

[https://docs.soliditylang.org/en/latest/assembly.html#memory-](https://docs.soliditylang.org/en/latest/assembly.html#memory-safety)

[safety::~text=byte%20scratch%20space%3A-,open%20in%20Remix,0%2C%20returndatasize\(\)\)%0A%20%20revert\(0%2C%20retuOn%20the%20other](https://docs.soliditylang.org/en/latest/assembly.html#memory-safety)

Mitigation

```
assembly ("memory-safe") {
    let p := mload(0x40)
    returndatacopy(p, 0, returndatasize())
    revert(p, returndatasize())
}
```

DepositApproved

[https://github.com/lifinance/VM-External-Audit-](https://github.com/lifinance/VM-External-Audit-Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/DepositApproved.sol#L20-L38)

[Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/DepositApproved.sol#L20-L38](https://github.com/lifinance/VM-External-Audit-Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/DepositApproved.sol#L20-L38)

```
assembly {
    // Batch both calls to reduce overhead
    let ptr := mload(0x40)

    // Prepare allowance call: allowance(address,address)
    mstore(ptr, 0xdd62ed3e00000000000000000000000000000000000000000000000000000000)
    mstore(add(ptr, 0x04), caller())
    mstore(add(ptr, 0x24), address())

    let success := staticcall(gas(), token, ptr, 0x44, ptr, 0x20)
    if and(success, eq(returndatasize(), 0x20)) { approvedAmount := mload(ptr) }

    // Prepare balanceOf call: balanceOf(address)
    mstore(ptr, 0x70a0823100000000000000000000000000000000000000000000000000000000) /// @audit after this the
memory expansions won't be inline with 32 anymore
    mstore(add(ptr, 0x04), caller())

    success := staticcall(gas(), token, ptr, 0x24, ptr, 0x20)
    if success { userBalance := mload(ptr) } /// @audit no contract check
}
```

The code writes to the FMP without updating it

Mitigation

Update the FMP by 68 bytes at least (you could increase by 96 if you wish to keep it word aligned)

Q-03 Gotcha: Transferring tokens will always require checking the return value

Impact

Because of how the Virtual Machine works

The lack of an `extcodesize` opcode

The VM is unable to perform a `safeTransfer` to recipients

This could cause no-op calls to non existing contracts to be interpreted as valid operations

This may also cause tokens that return `false` on transfer failure, to not cause a `revert`.

Due to this you may be forced to:

- Check that the contract exists (via another contract or by adding the `extcodesize` opcode)
- Check that a transfer returned `true` or nothing (but the contract exists)

Because the VM is assumed to be used in a nonReentrant context by a single user, then in most cases this should be fine.

However, for specific edge cases, this may result in total loss of funds:

- User believe they will receive at least X tokens
- They receive $X - 1$ tokens
- The transfer of X tokens is executed
- The token returns `false` instead of `reverting`
- The $X - 1$ tokens are in the VM lost to the first front-runner

Mitigation

Consider adding a `safeTransfer` helper to reduce gas cost and reduce the number of offchain checks that will have to be enforced

Q-04 `depositApproved` can return the incorrect amount for FoT and stETH like tokens

Impact

Some tokens charge a fee on transfer.

stETH is a rebasing token that internally uses shares, moving shares is subject to rounding errors.

Both vectors can lead the call to `depositApproved` to return an incorrect amount of tokens as the exact amount help by the VM will be lower.

Mitigation

The impact is fairly low and unlikely, and can be mitigated by performing a call to `balanceOf`

It's worth noting that with these tokens the user may receive even less

Q-05 `depositApproved` can return 0 on non existent tokens

Impact

The relevant code for `depositApproved` looks as follows:

<https://github.com/lifinance/VM-External-Audit-Quote/blob/a885743ad1e9381bf739b470be2a760805cb3c69/src/DepositApproved.sol#L16-L38>

```
IERC20 token = IERC20(depositCmd.token);

uint256 approvedAmount;
uint256 userBalance;
assembly {
    // Batch both calls to reduce overhead
    let ptr := mload(0x40)

    // Prepare allowance call: allowance(address,address)
    mstore(ptr, 0xdd62ed3e00000000000000000000000000000000000000000000000000000000)
    mstore(add(ptr, 0x04), caller())
    mstore(add(ptr, 0x24), address())

    let success := staticcall(gas(), token, ptr, 0x44, ptr, 0x20)
    if and(success, eq(returndatasize(), 0x20)) { approvedAmount := mload(ptr) }

    // Prepare balanceOf call: balanceOf(address)
    mstore(ptr, 0x70a0823100000000000000000000000000000000000000000000000000000000)
    mstore(add(ptr, 0x04), caller())

    success := staticcall(gas(), token, ptr, 0x24, ptr, 0x20)
    if success { userBalance := mload(ptr) } /// @audit no contract check
}
```

The code doesn't have an `EXTCODESIZE` check, meaning that: `success` can be `true` on non existent contracts.

POC

```
function test_callNonExistent() public {
    address token = 0x0000000000000000000000000000000000000000000000000000000000000000;
    uint256 userBalance;

    assembly {
        let ptr := mload(0x40)
        let success := staticcall(gas(), token, ptr, 0x24, ptr, 0x20)
        // Canary: Success and 0 will result in 1 so we know it's not a fluke nor a default value
        if success { userBalance := add(mload(ptr), 1) }
    }

    console.log("userBalance", userBalance);
}
```

Mitigation

Either add an `EXTCODESIZE` to the function or ensure the rest of the sequence will perform that check.

A-01 Suggested Next Steps

Maintain the Invariant Suite

The Invariant Suite has many limitations wrt state space exploration

It is best viewed as a integration tester

Given a specific integration (e.g. a Vault or a Bridge), you can fuzz those interactions fairly easily with a variety of tokens and actors.

The key limitation for the Suite are that it's pretty hard for it to explore specific states, (e.g. specific combinations for blueprints) as well as multiple sequences of non trivial calls (which make some memory bug hard to surface).

You can convert broken repros to Foundry Unit tests using this tool: <https://getrecon.xyz/tools/medusa>

You can read about our framework for Invariant Testing here:
https://book.getrecon.xyz/writing_invariant_tests/chimera_framework.html

Maintain the Halmos Tests and Integrate them to your main repo

Halmos tests can be used alongside Foundry tests to give you stronger assurances

Halmos effectively makes many unit tests and specific scenarios proven with a higher level of certainty

NOTE: Halmos tests will ignore reverts, foundry reverts should be considered false positives.

You can convert Halmos counterexamples to Foundry unit tests using this tool:
<https://getrecon.xyz/tools/halmos>

Extend Halmos based on edge cases generated by the custom fuzzing script

<https://github.com/Recon-Fuzz/li-fi-bf>

Because this script can generate new combinations, it has the highest change to help you identify new issues.

Once a problematic combination is found, it should be very simple to port that over to Halmos.

Suggested next engagements

We agree that the next step should be an Audit, focused on line by line low level code review. With a emphasis on the assembly components as well as ensuring no memory taint is possible (we highlighted an issue with the FMP, but were unable to escalate it's impact).

Additionally the audit should focus on the compositional risk that comes from integrating the VM in the specific ways we have discussed (Called exclusively by the GenericSwapFacet, with a Reentrancy guard), against the targets you plan to integrate.

A-02 Integration risks inspite of presence of slippage check when an attacker can regain access

Impact

Assuming:

- SwapFacet has no extra assets
- SwapFacet will perform an assertion for a minOut
- VirtualMachine cannot be reentered as it has a reentrancy guard
- VirtualMachine can only be called by the SwapFacet

Then there is a latent risk when the asset out is a ERC4626 Vault or some wrapper that can be reset.

This POC is purposefully a worst case scenario, however less impactful cases should be feasible.

POC

Assume we can regain control of the execution Assume the end asset of the sequence of swaps is an ERC4626 Assume we can reset it's PPFS by redeeming 100% of the tokens (default behaviour, we're unlikely to ever hold 100% of the token, hence this is an extreme scenario)

Assume the vault Price Per Share is 2e18 (2e18 assets give you 1e18 shares)

The user wants to deposit 2e18 assets, they expect to receive 1e18 shares

- The attacker gains control
- Reset the PPS
- Performs the donation attack to cause the users deposit to result in 0 shares minted
- Resets the PPS again
- Mints at 1:1
- Sends the 1:1 to the user (half the underlying value since they receive 1e18 shares a 1e18 pps instead of 2e18 pps)
- The user lost half the value from their deposit
- The attacker got the rest
- The assertion passes

Rationale

Note that this attack is feasible for ERC4626 from OZ as well as Solmate.

Other attacks scenario can most likely be found if we were to spend more time.

The point is to highlight that a simple balance check may not be sufficient for all cases, and should be assessed on a case by case basis.

Mitigation

A high level mitigation to this is to ensure that tokens cannot be reset and they are donation resistant. Euler vaults for example are Donation Resistant, however they can be reset (given that all of the supply can be collected to one user)

An alternative mitigation (that would work for this ERC4626 case) is to check the previewRedeem of the shares minted, if the previewRedeem is consistent with should have been deposited then the attack wasn't effective.

You should apply this type of reasoning to other types of assets as well (e.g. ERC7540), for these vaults `previewRedeem` will always revert.

Therefore for most integrations you'll have to perform DD before allowing an integration to be added.

A-03 Incomplete list of Reentrancy Techniques

- Native token has hooks: <https://medium.com/immunefi/a-poc-of-the-hundred-finance-heist-4121f23a098>
- Token is ERC777 or ERC677
- Token has tax that causes a swap to arbitrary address
- Value is sent to receiver / contract at any step except the last
- Friendtech tokens (fees sent to specific address)
- Malicious payload in VM (injected via FE)

A-04 RPN Math Gotchas

Impact

The key risks tied to using the RPN math as is is the inability to have explicit rounding directions

Many DeFi integrations have small and sometimes big integrator gotchas tied to rounding directions.

A common example is that to `withdraw` certain amounts of assets from a vault, we can't just compute the `totalShares / totalAssets`, instead we need to round the result of this division up in order to determine the shares that will be burned.

In most cases, where the integration target is liquid and not manipulatable, the difference is negligible.

In some cases, either for illiquid targets or when an attacker can re-enter, this difference can be crucial and cause severe losses.

Give the wide range of integrations that Li.Fi is aiming to support, adding divisions with explicit rounding should reduce value lost to MEV as well as prevent gotchas in risky integrations.

POC

A user needs to perform 2 operations and as part of 1 they need to withdraw assets from a vault. They would need to compute: `shares = assets.mulDivUp(supply, totalAssets())` They can either compute `assets * supply / totalAssets + 1` Or round it down to `assets * supply / totalAssets`

Based on the value of that wei of a share this can be a negligible difference (in most case) or an immense one (manipulate shares, low total supply vaults)

Mitigation

- Ensure the targets don't require math with rounding, or they are liquid enough to make a rebasing attack not feasible
- Consider adding a `MulDivUp` opcode
- Consider adding higher precision math (this is already achievable without needing changes)

A-05 Reflections on using the VM in production

Preamble

- The VM will be deployed and there will be a `onlyOwner` check for the Li.Fi diamond
- The VM will never hold user allowances for normal operation
- The VM will be able to receive arbitrary data from the Diamond
- The Diamond doesn't enforce Reentrancy protection in its `GenericSwapFacet`.

Impacts

I believe the key impact tied to this config is shared with the current `GenericSwapFacet` the facet could be reentered during a batch swap and funds on the contract could be stolen.

The main difference is that the `GenericSwapFacet` uses a list of valid targets to determine whether the call can be performed.

In contract, the VM will allow arbitrary operations in this possibly reentrant context

We assume that the VM would always a set of Assertions as the last set of commands.

This would push all the safety checks at this level.

Initial mitigations

With the presence of a reentrancy guard, the following invariant (TODO: CONFIRM) should be held:

At any time, the VM is executing exclusively the commands stemming from the first level depth of calls in the transaction, meaning there is no additional context in which the VM is executing.

In lack of a reentrancy guard, we cannot maintain this invariant.

There we must assume that every operation performed by the VM may be followed or preceded by another set of operations being performed by an attacker.

This opens up (TODO: Discuss further), to attacks such as the one found on uniswapX:

https://kebabsec.xyz/posts/critical_vulnerability_in_uniswapx/

If the assertion protecting the code are the only available, then all an attacker would have to do is find bypasses around said assertions.

TODO: Discuss implementation details on swap & transfer and how to bypass them

A naive implementation of a swap would perform a call from the VM to a swapper with some assets. And expect that the assets are received by the user. The assertion would enforce that the recipient funds have increased to a certain amount (as an exact amount could be grieved via a small donation).

In case of a reentrant call the check would be bypassed for users for which some other operation is being performed, as the assets (in the VM), would be stolen, and then some other operation (e.g. cross-chain minting, donation of sufficient assets) would happen.

(The same identical issue applies to performing the balance assertion to assert that the VM has sufficient assets, although it makes any attack even more likely)

Transferring of assets

A stronger solution would be to enforce that the assets are in the VM as the penultimate operation and then they are transferred to the correct recipient.

Enforcing that a certain amount of assets is transferred, when the asset is a base token (e.g. USDC, USDT, etc..) would ensure that the correct amount of tokens is transferred.

This could protect against many attacks.

However it would: Enforce that each swap is done by:

- Swapping to Out
- Transferring minOut (as to enforce the value transfer has happened)
- Transferring the remaining assets (as not to lose them)

This effectively doesn't remove the risk, but rather reduces it down to the delta between the actual value out vs minOut.

Additional edge case in transferring of assets

A direct transfer of assets would enforce that certain value was paid exclusively for assets that do not have a floating value. For tokens such as ERC4626 vaults, a certain quantity doesn't guarantee a specific value

The amount of checks necessary to ensure that the vault value is adequate would be tedious, implementation dependent and like grievable

Mitigation

The VM should use a Reentrancy Guard along with ensuring it can only be called by the Li.Fi diamond As agreed the VM should have no allowances at any time It may be best to transfer assets to the VM for all operations for the reasons detailed above

A-06 Handpicked Chain Specific Integration Risks

Executive Summary

This is the list of chains we were provided.

We attempt to note as many gotchas we can think of.

This list is incomplete, and should be extended by other auditors

This type of chain specific gotchas, as well as integrator specific gotchas are a good reason why a Security Contest or Bug Bounty could be helpful right before you deploy.

List with comments

1. Ethereum
2. BSC
3. Arbitrum
4. Base
5. Blast Should read the docs around how ETH and USDB work

- <https://docs.blast.io/about-blast>

TODO: REBASE ASSETS

Rebasing should be fine when moving assets from users It may cause up to 1 share of losses to the VM itself, but only for cross chain deposits targeting it. You may want to

MED: Ability to steal all gas

The first user can call the precompile and set themselves as the governor for the contract

This will allow them to collect all the gas fees

7. Avalanche
8. Polygon

9. Scroll

<https://docs.scroll.io/en/developers/ethereum-and-scroll-differences/>

The only thing I can think of is a grief that can be done by spamming precompiles, these txs would be dropped by the sequencer and make it hard to debug why.

But this would be mostly a self-rekt by the user.

10. Optimism

<https://docs.optimism.io/stack/differences>

Allow for forced inclusion (TX triggered from L1)

Aliasing applies to SC on L1 So the caller would be the aliased address Should not apply directly

May apply if someone tries to perform a L2 tx from a force included tx from L1

<https://docs.optimism.io/stack/transactions/forced-transaction>

11. Linea

12. zkSync -> Known issues with Addresses but most importantly huge Compiler risk

zkSync uses a separate compilation pipeline. This means you should not consider the results of this fuzzing engagement as applicable to the zkSync bytecode.

Famously AAVE has encountered a nasty compiler bug: <https://governance.aave.com/t/bgd-aave-v3-zksync-activation-issue-report/18819> <https://x.com/zkSyncDevs/status/1829110498471621003>
<https://x.com/zksync/status/1827850387950318071>

13. Polygon zkEVM

<https://docs.polygon.technology/zkEVM/overview/#security-measures>

Marked as deprecated in 2026.

Seems to use a compilation pipeline that is equivalent to mainnet.

15. Gnosis

Every token bridged by the canonical bridge is an ERC677 with hooks

<https://medium.com/immunefi/a-poc-of-the-hundred-finance-heist-4121f23a098#:~:text=The%20official%20bridge,whatever%20it%20likes>

I'm not 100% sure if this is still the case

16. Fantom

Fee Monetization: <https://docs.soniclabs.com/funding/fee-monetization/apply>

Seems to be offchain Not sure if anything can be done to grief onChain

I recommend talking to a BD from Fantom / Sonic

18. Moonriver

Not familiar

54. Berachain

The native AMM on Berachain is a Balancer V2 fork. Kankodu made a disclosure that non-existing tokens could be transferred in the vault to create "ghost credits"

https://mirror.xyz/0x38F1416B9Ed3a5DA9C12c56cb4F74D9564844728/iv9_q74rSlK7gbvbJAECuDIbzfUtrSCO6mSWIHPskKI

This is something that the Berachain team is actively monitoring and you should as well.

57. HyperEVM

The ability to call the precompiles (which afaik are used for trading), may cause some gotcha

<https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/hyperevm/interacting-with-hypercore>

Superchain

The same considerations as Optimism apply to these chains

23. Mode

24. Lisk

- 25. Unichain
- 26. Soneium
- 27. Swellchain
- 28. Celo
- 29. World Chain
- 30. Ink
- 31. BOB

Skipped

Due to a lack of experience and familiarity with these chains, I have no comment on them.

- 20. Moonbeam
- 21. FUSE
- 22. Boba
- 23. Metis
- 24. Aurora
- 25. Sei
- 26. Immutable zkEVM
- 27. Flare
- 28. Vana
- 29. Gravity
- 30. Taiko
- 31. opBNB
- 32. Corn
- 33. Lens
- 34. Cronos
- 35. Abstract
- 36. Rootstock
- 37. Apechain
- 38. Etherlink
- 39. XDC
- 40. Mantle
- 41. Superposition
- 42. Katana
- 43. Kaia
- 44. Plume

Additional Services by Recon

Recon offers:

- Audits powered by Invariant Testing - We'll write your invariant tests then perform an audit on your code.
- Cloud Fuzzing as a Service - The easiest way to run invariant tests in the cloud - Ask about Recon Pro.
- Invariant Tests writing - An engineer will write Chimera based Invariant Tests on your codebase.